



Module 3

CPU Scheduling and Deadlocks

3.1 Basic Concepts

In a computer with one CPU core, only one process can run at a time. Other processes must wait until the CPU is free. Multiprogramming is used to make sure the CPU is always doing some work, so its time is not wasted. A process runs until it needs to wait—for example, for input/output (I/O) to finish. In a simple system, the CPU would just sit idle during this wait. With multiprogramming, instead of letting the CPU stay idle, the operating system switches the CPU to another process. When that process also has to wait, the CPU is again given to a different process. This way, the CPU is kept busy all the time. In multicore systems, the same idea applies, but across all cores.

This scheduling of processes is one of the main jobs of an operating system. Since the CPU is one of the most important resources, scheduling it well is central to operating system design.

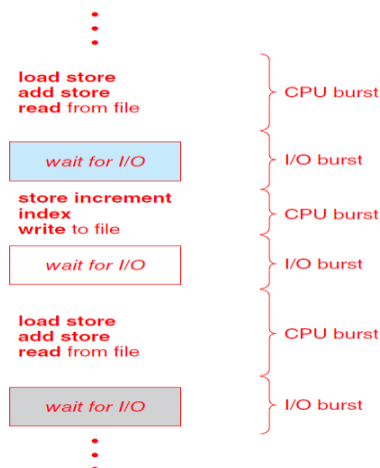


Figure 3.1 Alternating sequence of CPU and I/O bursts.

3.1.1 CPU-I/O Burst Cycle

- CPU scheduling works because of a common behavior in processes: execution alternates between CPU work and I/O waiting.
- A process starts with a CPU burst (performing computations).
- This is followed by an I/O burst (waiting for input/output).
- After that, another CPU burst occurs, then another I/O burst, and so on.
- Finally, the last CPU burst ends when the process issues a system call to terminate execution.

Studies of CPU burst times show a clear pattern:

- Most processes have many short CPU bursts and only a few long bursts.
- This distribution is usually exponential or hyper exponential in shape.
- I/O-bound programs → many short CPU bursts.
- CPU-bound programs → fewer but longer CPU bursts.

This pattern is important because CPU scheduling algorithms are often designed with this distribution in mind.

3.1.2 CPU Scheduler

When the CPU becomes idle, the operating system must choose a process from the ready queue to run next. This job is handled by the CPU scheduler, which selects one process from those that are in memory and ready to execute, and then allocates the CPU to it.

The ready queue does not always follow a simple first-in, first-out (FIFO) order. Depending on the scheduling algorithm, it can be organized as:

- A FIFO queue
- A priority queue
- A tree structure
- Or even an unordered linked list

Conceptually, all processes in the ready queue are just waiting for their turn to use the CPU. Each entry in this queue is usually represented by a process control block (PCB), which stores information about the process.

The burst duration (length of the CPU burst) of each process plays a key role in deciding how the scheduler chooses among them.

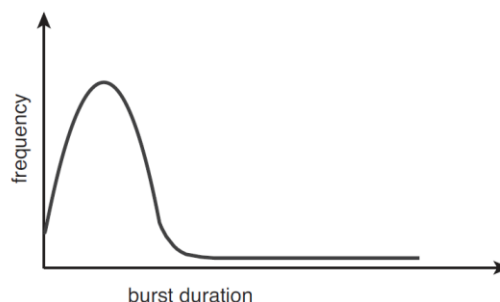


Figure 3.2 Histogram of CPU-burst durations.

3.1.3 Pre-emptive and Non-preemptive Scheduling

CPU scheduling decisions happen in four situations:

1. A process moves from running → waiting (e.g., an I/O request or waiting for a child process).
 2. A process moves from running → ready (e.g., when an interrupt occurs).
 3. A process moves from waiting → ready (e.g., I/O is finished).
 4. A process terminates.
- In cases 1 and 4, the CPU has no choice—the process can't continue, so another process must be scheduled.
 - In cases 2 and 3, the operating system can decide whether to continue the current process or switch to another.

This leads to two scheduling types:



1. Non-preemptive (or cooperative) scheduling:

- CPU is given to a process, and it keeps it until it either finishes or voluntarily moves to waiting.
- Simpler, but less efficient.

2. Pre-emptive scheduling:

- The CPU can be taken away from a process before it finishes its burst.
- Used in almost all modern OS (Windows, Linux, macOS, UNIX).
- But it can cause race conditions if processes share data (e.g., one is updating data when it gets preempted, leaving the data inconsistent).

Kernel Design Impact:

- A non-pre-emptive kernel lets system calls finish or a process block before switching. This makes the kernel simpler but not suitable for real-time systems.
- A pre-emptive kernel can interrupt even in kernel mode. To avoid problems, it uses synchronization tools like mutex locks to protect shared data.
- Most modern OS kernels are fully pre-emptive.

Because interrupts can happen at any time and the kernel cannot simply ignore them, parts of the code that are affected by interrupts must be protected. The operating system must usually accept interrupts so that important events are not missed (e.g., input being lost or output being overwritten).

To avoid multiple processes accessing the same critical code at once, the OS temporarily disables interrupts when entering such a section and reenables them when leaving. These interrupt-disabled sections are very short and contain only a few instructions.

3.1.4 Dispatcher

The dispatcher is the part of the operating system that gives control of the CPU to the process chosen by the scheduler. It is responsible for:

- Switching context from one process to another,
- Changing to user mode,
- Jumping to the correct instruction in the selected process so it can continue running.

Since the dispatcher runs during every context switch, it must be very fast. The time it takes to stop one process and start another is called the dispatch latency.

The frequency of context switches depends on the system's workload and scheduling policy. On Linux, you can check this system-wide count using the `vmstat` command, which shows how many context switches are happening.

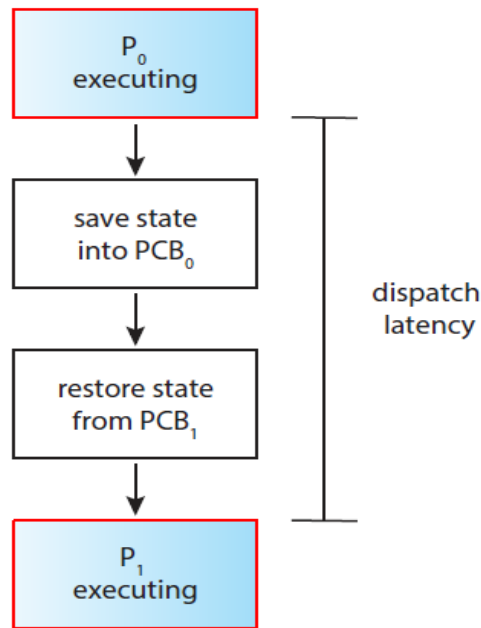


Figure 3.3 The role of the dispatcher.

Over two 1-second intervals, the system recorded the following context switches:

- Average since boot: 24 per second
- Last second: 225 context switches
- Second before that: 339 context switches

We can also see context switch details for a specific process using the /proc file system. For example, the command:

```
cat /proc/2166/status
```

might show:

voluntary ctxt switches 150

nonvoluntary ctxt switches 8

This means:

- Voluntary context switches → The process gave up the CPU on its own, usually because it was waiting for something (like I/O).
- Nonvoluntary context switches → The CPU was taken away from the process, either because its time slice ended or a higher-priority process needed the CPU.

3.2 Scheduling Criteria

Different CPU scheduling algorithms behave differently, and the choice of algorithm can benefit one type of process more than another. To decide which algorithm is best for a given situation, we compare them using certain criteria:

1) CPU Utilization → Keep the CPU as busy as possible.

- Ideal: 0–100%
- Real systems: about 40% (light load) to 90% (heavy load).



- Can be checked using commands like top in Linux/macOS/UNIX.

2). Throughput → The amount of work done, measured as the number of processes completed per unit time.

- Throughput → Number of processes finished per unit of time.
- For long processes: may be 1 process every few seconds.
- For short processes: may be tens of processes per second.

3). Turnaround Time → Total time taken for a process to finish.

- From the time it is submitted → until it completes.
- Includes: waiting in the ready queue + CPU execution time + I/O time.

4). Waiting Time → The total time a process spends only waiting in the ready queue.

- Scheduling algorithms mainly affect this measure.

5). Response Time → For interactive systems, this is key.

- Time from submission of a request → until the first response is produced.
- It measures how quickly a system reacts, not how long the full output takes.

Scheduling Algorithm Optimization Criteria

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time

Other considerations:

- Sometimes, minimizing the maximum response time (worst case) is more important than optimizing the average.
- In interactive systems (like PCs), users prefer consistent and predictable response times rather than faster but highly variable times.
- For simplicity, when studying algorithms, we often use examples with one CPU burst per process and compare them using the average waiting time.

3.2.1 Scheduling Algorithms

Material is available in another notes

3.2.6 Multilevel Queue Scheduling

1. Basic Idea



- Instead of having all processes in a single ready queue, we divide them into multiple queues.
- Each queue can represent either:
 1. Priority levels (separate queue for each priority), or
 2. Process types (e.g., real-time, system, interactive, batch).
- Each queue may have its own scheduling algorithm.

2. How It Works

- Highest priority queue is always served first.
- If a queue is empty → the scheduler looks at the next lower queue.
- Example: A real-time queue may have absolute priority over all other queues.
- If a new high-priority process arrives, it preempts the currently running lower-priority process.

3. Example (4 Queues, in order of priority):

- Real-time processes (highest)
- System processes
- Interactive processes
- Batch processes (lowest)
 - 1). A batch process will only run if all higher queues are empty.
 - 2). If an interactive process arrives while a batch is running → the batch is pre-empted.

4. Combination of Scheduling

- Each queue can use a different scheduling algorithm:
- Foreground (interactive) → Round-Robin (RR)
- Background (batch) → First-Come, First-Served (FCFS)
- This allows the system to balance different needs (fast response for interactive tasks, fairness for batch tasks).

5. Time-Slicing Among Queues

- Instead of absolute priority, the CPU can be shared between queues.
- Example:

Foreground queue → gets 80% CPU time (RR).

Background queue → gets 20% CPU time (FCFS).

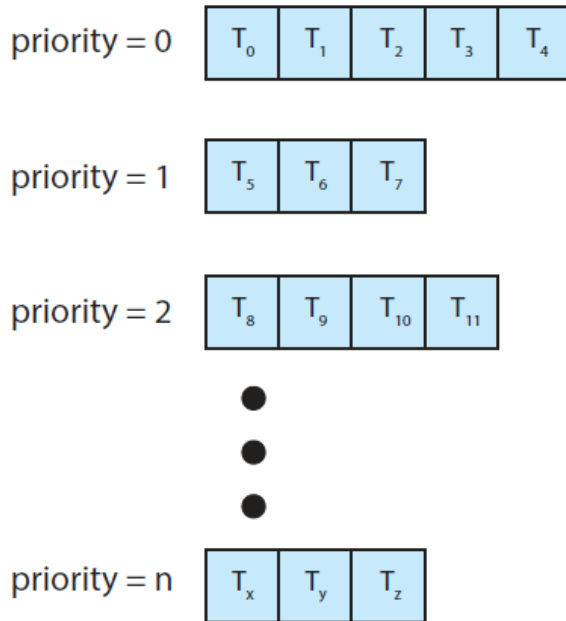


Figure 3.7 Separate queues for each priority.

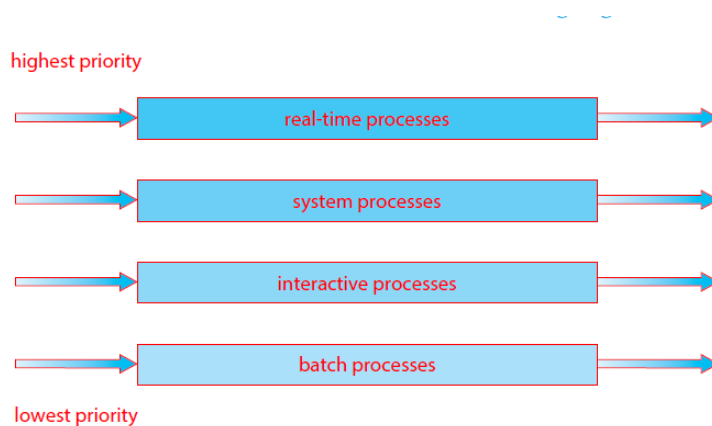


Figure 3.8 Multilevel queue scheduling.

3.2.7 Multilevel Feedback Queue Scheduling

In a normal multilevel queue scheduling, once a process is put into a queue (for example, foreground or background), it stays there permanently. Processes don't move between queues. This method is simple and has low overhead, but it's not flexible.

In contrast, multilevel feedback queue scheduling allows processes to move between queues. The goal is to separate processes based on how much CPU time they need:

- If a process uses too much CPU, it is pushed to a lower-priority queue.
- Short, interactive, or I/O-bound processes (which use little CPU) stay in higher-priority queues.



- If a process waits too long in a lower queue, it can be moved up (called aging) so it doesn't starve.

Example

Imagine there are 3 queues:

Queue 0 → highest priority

Queue 1 → medium priority

Queue 2 → lowest priority

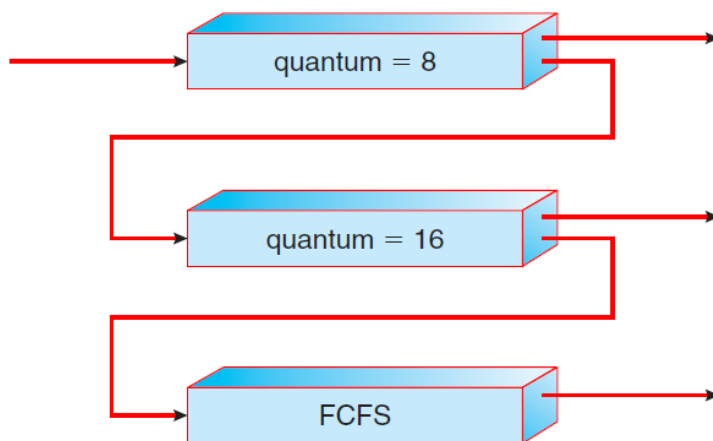


Figure 3.9 Multilevel feedback queues.

The scheduler works like this:

Queue 0 is always checked first. Only if it's empty will Queue 1 be used. Queue 2 runs only if both 0 and 1 are empty.

If a process enters the system, it starts in Queue 0 with 8 ms of CPU time (time quantum). If it doesn't finish, it moves to Queue 1.

In Queue 1, it gets 16 ms. If it still doesn't finish, it moves to Queue 2.

Queue 2 works in First-Come, First-Served (FCFS) order and runs only when the other two are empty.

This means:

- Processes that need ≤ 8 ms get the CPU quickly.
- Processes needing between 8–24 ms also get CPU but with slightly less priority.
- Long processes eventually end up in Queue 2 and run in FCFS order.



Key Parameters of Multilevel Feedback Queue/To set up this scheduler, we must define:

- How many queues there are.
- What scheduling method each queue uses.
- When to promote a process to a higher queue.
- When to demote a process to a lower queue.

Where a new process starts (which queue).

This makes it the most flexible and powerful CPU scheduling method, but also the most complex to design.

3.3 Process Synchronization

Background

We already know that processes can run concurrently or in parallel.

In concurrent execution (Section 3.2.2), the CPU scheduler quickly switches between processes, giving the illusion that they are running at the same time. Because of this, a process may only run for a short while before being interrupted, and the CPU may be given to another process. In fact, a process can be stopped at any point in its instruction sequence so another process can run.

In parallel execution, two or more processes truly run at the same time on different CPU cores. In this chapter, we look at how both concurrent and parallel execution can cause problems when multiple processes share the same data, and how such situations can affect the correctness and integrity of that shared data.

3.3.1 The Critical-Section Problem

When multiple processes run in a system, they may sometimes need to use shared data. Each process has a special part of its code, called the critical section, where it accesses or updates shared data.

The main rule is:

Only one process can be inside its critical section at a time. If one process is using the shared data, no other process should be allowed to enter its critical section.

The challenge, called the critical-section problem, is to design a way (a protocol) for processes to coordinate so they can safely share data.

To do this, each process must:

- Request permission before entering its critical section (this part is called the **entry section**).
- After finishing, signal that it is leaving (this part is called the **exit section**).
- The rest of the program (not related to shared data) is called the **remainder section**.



So, the general structure of a process looks like this:

Requirements for Solving the Critical-Section Problem

Any correct solution to the critical-section problem must follow three rules:

1. Mutual Exclusion

- If one process (P_i) is inside its critical section, then no other process can be inside its critical section at the same time.
- This ensures that shared data is not changed by two processes simultaneously.

2. Progress

- If no process is currently in its critical section, and there are some processes that want to enter, then the system must decide fairly and quickly which process will enter next.
- Only processes that are ready and waiting (not busy in their remainder section) can take part in this decision.
- The decision cannot be delayed forever.

```
while (true) {  
    entry section  
    critical section  
    exit section  
    remainder section  
}
```

Figure 3.10 General structure of a typical process.

3. Bounded Waiting

The third rule of the critical-section problem is bounded waiting.

- This means: once a process requests to enter its critical section, there must be a limit on how many times other processes can go ahead of it before it gets its turn.
- In other words, no process should have to wait forever — each process will eventually get its chance.



We assume that:

- Every process runs at some nonzero speed.
- But we cannot assume that processes run at the same speed (one process may run faster or **slower than another**).

Race Conditions in the Kernel

In an operating system, many kernel-mode processes may run at the same time. Kernel code often accesses shared data structures, and if mutual exclusion is not enforced, race conditions can occur.

Example 1: Open files list

The OS keeps a list of all open files. If two processes open files at the same time, they may both try to update this list. Without protection, the list could become corrupted.

Example 2: fork() and process IDs

When two processes (say P0 and P1) create child processes using the fork() system call, the kernel assigns each child a process identifier (PID). The variable next_available_pid stores the next unused PID.

- If two processes update this variable at the same time, they may both get the same PID.
- This creates a race condition, leading to incorrect or duplicate process IDs.

Other OS structures at risk:

- Memory allocation tables
- Process lists
- Interrupt handling mechanisms

It's the job of kernel developers to make sure these shared structures are protected against race conditions.

How to Solve (Single-Core Idea)

On a single-core system, the critical-section problem could be solved by simply disabling interrupts whenever a shared variable is being updated.

- This prevents the CPU from being interrupted mid-update.
- That way, no other process can interfere until the update is complete.

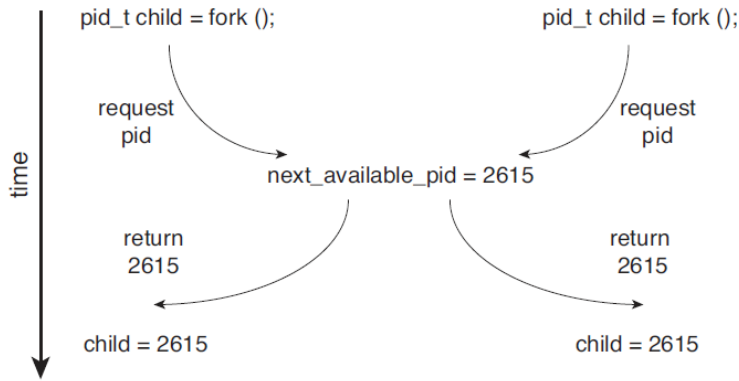


Figure 3.11 Race condition when assigning a pid.

Solving the Critical-Section Problem in Single- and Multi-Core Systems

On a single-core CPU, the critical-section problem can be solved by simply disabling interrupts while a shared variable is being updated.

- This ensures that the sequence of instructions runs without interruption.
- No other process can run during this time, so the shared variable cannot be unexpectedly changed.

However, this method is not practical in multiprocessor systems:

- Disabling interrupts must be communicated to all processors, which is slow and inefficient.
- This delay reduces system performance.
- It can also affect the system clock, which relies on interrupts to stay updated.

Two Approaches to Critical Sections in Kernels

1.Non-pre-emptive Kernel

- A process running in kernel mode cannot be interrupted.
- It runs until it either:
 - finishes kernel mode,
 - blocks, or
 - voluntarily gives up the CPU.
- This design avoids race conditions in kernel data, since only one kernel process is active at a time.

2.Pre-emptive Kernel

- A process running in kernel mode can be interrupted and replaced by another process.
- This makes design more complex, especially on multi-core (SMP) systems, where two kernel processes may run at the same time on different CPUs.
- Developers must carefully ensure that shared kernel data is protected against race conditions.



Why Use Pre-emptive Kernels?

Even though they are harder to design, pre-emptive kernels have important advantages:

- **Better responsiveness:** Processes don't have to wait indefinitely for a long kernel process to finish.
- **Real-time support:** Real-time processes can immediately pre-empt other processes, even if they are running in kernel mode.

Non-pre-emptive kernels are simpler and safer, but pre-emptive kernels are faster and better suited for real-time systems.

3.3.2 Peterson's Solution

Peterson's solution is a classic software method to solve the critical-section problem.

⚠ Note: On modern CPUs, due to the way low-level instructions like load and store are executed, Peterson's solution may not always work correctly. Still, it is very useful for learning, since it clearly shows how to design an algorithm that satisfies the three requirements:

1. Mutual Exclusion
2. Progress
3. Bounded Waiting

Key Idea

- Peterson's solution works for two processes only (say P_0 and P_1).
- These two processes take turns entering their critical section.
- Each process alternates between:
 1. Critical Section (where it accesses shared data)
 2. Remainder Section (where it does other work)

When describing process P_i , the other process is called P_j , where $j = 1 - i$.

- If $i = 0$, then $j = 1$.
- If $i = 1$, then $j = 0$.

Shared Data

Peterson's solution uses two shared variables:

- $flag[2]$ → an array where:
 1. $flag[i] = \text{true}$ means process P_i wants to enter its critical section.
- $turn$ → an integer variable that indicates whose turn it is to enter the critical section.

The combination of $flag[]$ and $turn$ ensures:

- Only one process can be in the critical section at a time (Mutual Exclusion).
- If both want to enter, they take turns fairly (Progress + Bounded Waiting).

Proving Peterson's Solution

To prove that Peterson's solution works, we must show that it satisfies the three rules:



1. Mutual Exclusion

a. A process P_i can enter its critical section only if:

- The other process P_j is not interested ($\text{flag}[j] == \text{false}$), or
- It is P_i 's turn ($\text{turn} == i$).

b. If both processes try to enter at the same time, then:

- Both will set their $\text{flag}[]$ to true.
- But turn can only be 0 or 1, not both.

c. This means only one process can pass the while-loop check.

d. Therefore, two processes cannot be in the critical section at the same time $\rightarrow$ Mutual Exclusion holds.

2. Progress

- A process P_i can be blocked only if:
 1. $\text{flag}[j] == \text{true}$ (the other process also wants to enter), and
 2. $\text{turn} == j$ (the other process's turn).
- If the other process P_j is not interested ($\text{flag}[j] == \text{false}$), then P_i can enter immediately.
- If both are interested, then one will enter (say P_j).
- After P_j finishes, it sets $\text{flag}[j] = \text{false}$, so P_i can then enter.

Thus, whenever the critical section is free, one of the waiting processes must get a chance to enter $\rightarrow$ Progress holds.

3. Bounded Waiting

- If P_i wants to enter, the worst case is:
 - 1. P_j enters once before P_i (because $\text{turn} == j$).
 - But when P_j re-enters, it must set $\text{turn} = i$.
 - This ensures P_i gets priority next time.
-
- So, P_i will not be postponed indefinitely $\rightarrow$ Bounded Waiting holds.

Limitation on Modern Systems

On modern CPUs, Peterson's solution is not always reliable because:

- To improve speed, processors and compilers may reorder read and write instructions.
- In single-threaded programs, this is fine (the final result is correct).
- But in multi-threaded programs with shared data, such reordering may cause unexpected results, breaking the correctness of Peterson's solution.

3.4.3 Mutex Locks



The hardware solutions to the critical-section problem (discussed earlier) are often too complex and not usable by normal application programmers. So, operating systems provide a software tool called a mutex lock.

- Mutex = short for Mutual Exclusion.
- It is used to protect critical sections and prevent race conditions.

How It Works

- Before a process enters its critical section, it must acquire the lock using the `acquire()` function.
- After leaving the critical section, it must release the lock using the `release()` function.
- This ensures that only one process at a time can be inside the critical section.

Internal Working

A boolean variable (say, `available`) is associated with the lock.

- If `available = true`, the lock is free → a process can successfully acquire it.
- If `available = false`, the lock is already taken → the requesting process is blocked until the lock becomes free.

```
while (true) {  
    acquire lock  
    critical section  
    release lock  
    remainder section  
}
```

Figure 5.11 Solution to the critical-section problem using mutex locks.
Contended vs. Uncontended Locks

Note:

- A lock controls access to a shared resource (like a variable or memory).
- If a thread attempts to acquire the lock and it is free, it succeeds immediately → this is an uncontended lock.
- If a thread tries to acquire the lock but another thread already holds it, the thread must wait (block) → this is a contended lock.

Disadvantage of Mutex Lock with Busy Waiting



1. In the basic mutex lock design, if a process wants the lock but it's already taken, the process keeps looping (checking repeatedly if the lock is free).
2. This is called busy waiting, because the process is "busy" doing nothing useful while it waits.
3. On a single CPU system, this is a big problem: one process is looping and wasting CPU cycles that could be given to another process.

Spinlocks

- A mutex lock that uses busy waiting is also called a spinlock.
- The word comes from the fact that the waiting process "spins" in a loop until the lock becomes available.

Pros and Cons

Cons:

- Wastes CPU cycles.
- Not efficient on single-core systems where only one process can run at a time.

Pros:

- No context switch is needed (switching CPU from one process to another), which saves time.
- On multicore systems, spinlocks can be useful if the lock is only held briefly:
- One thread spins on one core.
- Another thread quickly finishes its work on another core and releases the lock.

Semaphores

- Mutex locks are the simplest way to handle synchronization, but sometimes we need a more powerful tool. That's where semaphores come in.
- A semaphore (S) is basically an integer variable that helps processes coordinate their activities. Unlike a mutex, which only locks and unlocks, a semaphore can be used in more flexible ways.

```
wait(S) {  
    while (S <= 0)  
        ; // busy wait  
    S--;  
}
```

The definition of `signal()` is as follows:

```
signal(S) {  
    S++;  
}
```

Atomic Operations in Semaphores

- When processes use `wait(S)` and `signal(S)`, the value of the semaphore S must be changed atomically.
- Atomic means the whole operation happens at once, without being interrupted by any other process.

Why Atomicity Matters



Suppose two processes try to modify S at the same time. If the operation isn't atomic, they could both read the same value and overwrite each other, leading to errors (race conditions).

In wait(S):

- The process checks if $S \leq 0$.
- If not, it decreases S (S--).
- Both the test and the decrement must be done together, without interruption.

Types of Semaphores

1. Counting Semaphore

- Value can be any non-negative integer.
- Used when there are multiple identical resources.
- Example: If 5 printers are available, initialize the semaphore to 5.
 - 1). Every time a process wants a printer → it does wait() (count decreases).
 - 2). Every time a process finishes and releases a printer → it does signal() (count increases).
 - 3). If the count is 0, no printer is available → new processes must wait.

2. Binary Semaphore

- Value is only 0 or 1.
- Behaves just like a mutex lock (one process in the critical section at a time).
- Useful for mutual exclusion if a system doesn't provide mutex locks directly.

Using Semaphores for Synchronization

- Semaphores can also be used to control the order of execution between processes.

Example: Two processes P1 and P2, where statement S2 in P2 must run only after S1 in P1 has finished.

- Create a semaphore synch, initialized to 0.
- In P1, after executing S1, add:

```
S1;  
signal(synch);
```

In process P2, we insert the statements

```
wait(synch);  
S2;
```

Because synch is initialized to 0, P2 will execute S2 only after P1 has invoked signal(synch), which is after statement S1 has been executed.

Semaphore Implementation

To fix this, semaphores are designed so that:



1. In wait(S):

- If the semaphore's value is positive, the process continues as normal.
- If the value is 0 or less, instead of looping, the process is suspended.
- The suspended process is put into a waiting queue attached to the semaphore.
- Its state changes to waiting, and the CPU is given to another process by the scheduler.

2. In signal(S):

- When another process calls signal(S), one of the suspended processes is woken up.
- A wakeup() operation changes its state from waiting to ready.
- The process is then placed into the ready queue, waiting for the CPU.
- Depending on the CPU scheduling policy, it may run immediately or wait its turn.

```
typedef struct {  
    int value;  
    struct process *list;  
} semaphore;
```

Each semaphore has an integer value and a list of processes list. When a process must wait on a semaphore, it is added to the list of processes. A signal() operation removes one process from the list of waiting processes and awakens that process.

Now, the wait() semaphore operation can be defined as

```
wait(semaphore *S) {  
    S->value--;  
    if (S->value < 0) {  
        add this process to S->list;  
        sleep();  
    }  
}
```

and the signal() semaphore operation can be defined as

```
signal (semaphore *S) {  
    S->value++;  
    if (S->value <= 0) {  
        remove a process P from S->list;  
        wakeup(P);  
    }  
}
```

sleep() and wakeup()



- `sleep()` → suspends the process that calls it.
- `wakeup(P)` → resumes a process P that was previously suspended.
- Both are basic system calls provided by the operating system.

Semaphore Values

- In this implementation, semaphore values can become negative.
- If $S < 0$, then the absolute value of S tells us how many processes are waiting on that semaphore.
 - Example: $S = -3$ means 3 processes are in the waiting queue.
- This difference comes from the way the `wait()` operation is implemented:
- First it decrements S, then it tests the value.

3.4 Classic Problems of Synchronization

- When multiple processes (or threads) run at the same time, they often need to coordinate access to shared resources. If not managed properly, this can cause race conditions, deadlocks, or starvation.
- To study and test synchronization methods, computer scientists use a set of classic synchronization problems. These problems are like “benchmarks” if a solution works for them, it usually works for real-world concurrency issues too.

3.4.1. Bounded-Buffer Problem

- Also called the Producer–Consumer Problem.
- You have n buffers (each can hold one item).
- Two types of processes:
 - Producer → generates data and puts it into a buffer.
 - Consumer → takes data from a buffer and uses it.
- The problem: coordinate them so that:
 - The producer doesn't put data into a full buffer.
 - Consumer doesn't take data from an empty buffer.
 - Both don't interfere with each other (race conditions).

The producer and consumer processes share the following data structures:

```
int n;
```

```
semaphore mutex = 1;
```

```
semaphore empty = n;
```

```
semaphore full = 0;
```

Setup of the Bounded-Buffer Problem

There are n buffers, each buffer can hold one item.

Three semaphores are used:

1. **mutex** (binary semaphore, initialized to 1)

- Ensures only one process (producer or consumer) accesses the buffer at a time → prevents race conditions.



2. **empty** (counting semaphore, initialized to n)

- Keeps track of how many buffers are empty.
- Starts at n because all buffers are empty initially.

3. **full** (counting semaphore, initialized to 0)

- Keeps track of how many buffers are full.
- Starts at 0 because no items are produced yet.

```
while (true) {  
    . . .  
    /* produce an item in next_produced */  
    . . .  
    wait(empty);  
    wait(mutex);  
    . . .  
    /* add next_produced to the buffer */  
    . . .  
    signal(mutex);  
    signal(full);  
}
```

Figure 7.1 The structure of the producer process.

...

```
while (true) {  
    wait(full);  
    wait(mutex);  
    . . .  
    /* remove an item from buffer to next_consumed */  
    . . .  
    signal(mutex);  
    signal(empty);  
    . . .  
    /* consume the item in next_consumed */  
    . . .  
}
```

Figure 7.2 The structure of the consumer process.

3.4.2 The Readers–Writers Problem

- Suppose that a database is to be shared among several concurrent processes. Some of these processes may want only to read the database, whereas others may want to update (that is, read and write) the database. We distinguish between these two types of processes by referring to the former as readers and to the latter as writers. Obviously, if two readers access the shared data simultaneously, no adverse effects will result. However, if a writer and some other process (either a reader or a writer) access the database simultaneously, chaos may ensue. To ensure that these difficulties do not arise, we require that the writers have exclusive access to the shared database while writing to the database. This synchronization problem is referred to as the readers–writers



problem. The readers-writers problem has several variations, all involving priorities. The simplest one, referred to as the first readers-writers problem, requires that no reader be kept waiting unless a writer has already obtained permission to use the shared object. In other words, no reader should wait for other readers to finish simply because a writer is waiting. The second readers-writers problem requires that, once a writer is ready, that writer perform its write as soon as possible. In the solution to the first readers-writers problem, the reader processes share the following data structures:

- semaphore rw_mutex = 1;
- semaphore mutex = 1;
- int read count = 0;
- The binary semaphores mutex and rw_mutex are initialized to 1; read count is a counting semaphore initialized to 0. The semaphore rw_mutex is common to both reader and writer processes. The mutex semaphore is used to ensure mutual exclusion when the variable read count is updated. The read count variable keeps track of how many processes are currently reading the object. The semaphore rw_mutex functions as a mutual exclusion semaphore for the writers. It is also used by the first or last reader that enters or exits the critical section. It is not used by readers that enter or exit while other readers are in their critical sections.
-
-
- Two types of processes use it:
 - Readers → only read the database (don't change anything).
 - Writers → read and update (modify) the database.

```
while (true) {  
    wait(rw_mutex);  
  
    . . .  
    /* writing is performed */  
    . . .  
    signal(rw_mutex);  
}
```

Figure 7.3 The structure of a writer process.

1. Readers only (no problem):

If two or more readers access the shared data at the same time → no issues.

2. Writer involved (problem starts):

If a writer and another process (reader or writer) access the database together → errors/confusion may happen.

3. Rule needed:



- Writers must get exclusive access while writing.
- This is called the readers–writers problem (used to test synchronization techniques).

4. Two main variations:

First readers–writers problem:

- Readers should not wait unless a writer is already writing.
- A writer waiting should not block readers.

```
while (true) {
    wait(mutex);
    read_count++;
    if (read_count == 1)
        wait(rw_mutex);
    signal(mutex);

    . . .
    /* reading is performed */
    . . .
    wait(mutex);
    read_count--;
    if (read_count == 0)
        signal(rw_mutex);
    signal(mutex);
}
```

Figure 7.4 The structure of a reader process.

Second readers–writers problem:

- If a writer is waiting, it must be given access as soon as possible.
- No new readers are allowed to start until the writer finishes.

5. Starvation issue:

- In the first problem → writers may starve (because readers keep coming).
- In the second problem → readers may starve (because writers keep getting priority).

6. Other solutions:

- Variants exist that prevent starvation for both readers and writers.
- Example: starvation-free solutions for the second problem.

The code for a writer process is shown in Figure 7.3; the code for a reader process is shown in Figure 7.4. Note that, if a writer is in the critical section and n readers are waiting, then one reader is queued on `rw mutex`, and $n - 1$ readers are queued on `mutex`. Also observe that, when a writer executes `signal(rw mutex)`, we may resume the execution of either the waiting readers or a single waiting writer. The selection is made by the scheduler.



3.4.3 Dining-Philosophers Problem

Setup:

- Five philosophers sit around a circular table.
- Each philosopher has a chair.
- A bowl of rice is in the middle.
- Five chopsticks are placed between them (one between each pair).

Philosophers' life:

- They spend their time thinking or eating.
- While thinking → no interaction with others.
- When hungry → need two chopsticks (left and right) to eat.

Rules for chopsticks:

- A philosopher can pick up only one chopstick at a time.
- Cannot take a chopstick if a neighbor is already holding it.
- Only when both chopsticks are in hand → philosopher can eat.

After eating:

- Philosopher puts down both chopsticks.
- Goes back to thinking.

Why is this problem important?

- It's a classic synchronization problem in operating systems.
- Shows how processes sharing resources can lead to issues like:
 - 1). Deadlock (everyone waiting, no one eats).
 - 2). Starvation (some philosophers never get a chance to eat).
- Used to study concurrency control (managing multiple processes).

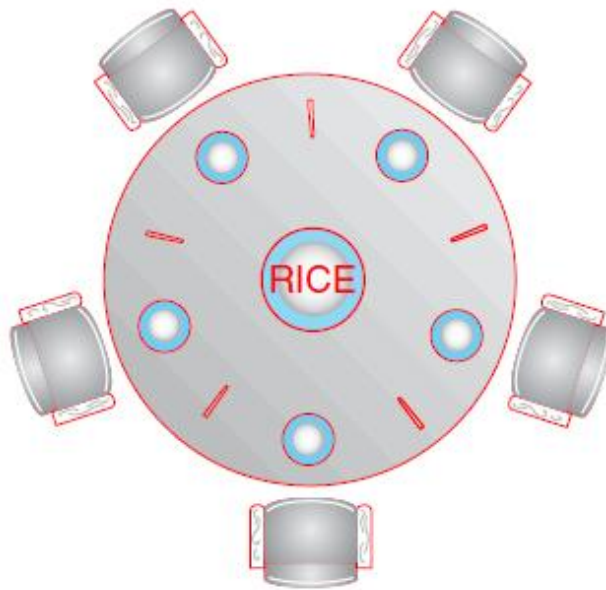


Figure 3.13 The situation of the dining philosophers.

Dining-Philosophers Problem – Solutions

1. Semaphore Solution

- Each chopstick = a semaphore, initialized to 1.
- A philosopher picks a chopstick → wait() operation.
- A philosopher releases a chopstick → signal() operation.
- Shared data:

```
semaphore chopstick[5]; // all initialized to 1
```

```
while (true) {  
    wait(chopstick[i]);  
    wait(chopstick[(i+1) % 5]);  
    . . .  
    /* eat for a while */  
    . . .  
    signal(chopstick[i]);  
    signal(chopstick[(i+1) % 5]);  
    . . .  
    /* think for awhile */  
    . . .  
}
```

Figure 7.6 The structure of philosopher *i*.



A philosopher tries to grab a chopstick by executing a `wait()` operation on that semaphore. She releases her chopsticks by executing the `signal()` operation on the appropriate semaphores. Thus, the shared data are semaphore chopstick[5]; where all the elements of chopstick are initialized to 1. The structure of philosopher *i* is shown in Figure 7.6.

Problem with this solution:

- Guarantees that neighbors won't eat at the same time.
- But can cause deadlock:

1. If all 5 philosophers pick up their left chopstick at the same time, all semaphores become 0.

2. Now, everyone waits for the right chopstick → stuck forever.

Possible remedies:

Several possible remedies to the deadlock problem are the following:

- Allow at most four philosophers to be sitting simultaneously at the table.
- Allow a philosopher to pick up her chopsticks only if both chopsticks are available (to do this, she must pick them up in a critical section).
- Use an asymmetric solution—that is, an odd-numbered philosopher picks up first her left chopstick and then her right chopstick, whereas an even numbered philosopher picks up her right chopstick and then her left chopstick.



❖ **Deadlocks:**

- Deadlocks;
- System model;
- Deadlock characterization;
- Methods for handling deadlocks;
- Deadlock prevention;
- Deadlock avoidance;
- Deadlock detection and recovery from deadlock.

MODULE 3

DEADLOCKS

A process requests resources, if the resources are not available at that time, the process enters a waiting state. Sometimes, a waiting process is never again able to change state, because the resources it has requested are held by other waiting processes. This situation is called a **Deadlock**.

SYSTEM MODEL

- A system consists of a finite number of resources to be distributed among a number of competing processes. The resources are partitioned into several types, each consisting of some number of identical instances. Memory space, CPU cycles, files, and I/O devices are examples of resource types.
- A process must request a resource before using it and must release the resource after using it. A process may request as many resources as it requires carrying out its designated task. The number of resources requested may not exceed the total number of resources available in the system.

Under the normal mode of operation, a process may utilize a resource in only the following sequence:

1. **Request:** The process requests the resource. If the request cannot be granted immediately, then the requesting process must wait until it can acquire the resource.
2. **Use:** The process can operate on the resource.
3. **Release:** The process releases the resource.

A set of processes is in a deadlocked state when every process in the set is waiting for an event that can be caused only by another process in the set. The events with which we are mainly concerned here are resource acquisition and release. The resources may be either physical resources or logical resources

To illustrate a deadlocked state, consider a system with three CD RW drives.

Suppose each of three processes holds one of these CD RW drives. If each process now requests another drive, the three processes will be in a deadlocked state.

Each is waiting for the event "CD RW is released," which can be caused only by one of the other waiting processes. This example illustrates a deadlock involving the same resource type.

Deadlocks may also involve different resource types. For example, consider a system with one printer and one DVD drive. Suppose that process P_i is holding the DVD and process P_j is holding the printer. If P_i requests the printer and P_j requests the DVD drive, a deadlock occurs.

DEADLOCK CHARACTERIZATION

Necessary Conditions

A deadlock situation can arise if the following four conditions hold simultaneously in a system:

1. **Mutual exclusion:** At least one resource must be held in a non-sharable mode, that is, only one process at a time can use the resource. If another process requests that resource, the requesting process must be delayed until the resource has been released.
2. **Hold and wait:** A process must be holding at least one resource and waiting to acquire additional resources that are currently being held by other processes.
3. **No preemption:** Resources cannot be preempted; that is, a resource can be released only voluntarily by the process holding it, after that process has completed its task.
4. **Circular wait:** A set $\{P_0, P_1, \dots, P_n\}$ of waiting processes must exist such that P_0 is waiting for a resource held by P_1 , P_1 is waiting for a resource held by P_2 , $\dots$, P_{n-1} is waiting for a resource held by P_n and P_n is waiting for a resource held by P_0 .

Resource-Allocation Graph

Deadlocks can be described in terms of a directed graph called **System Resource-Allocation Graph**

The graph consists of a set of vertices V and a set of edges E . The set of vertices V is partitioned into two different types of nodes:

- $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the active processes in the system.
- $R = \{R_1, R_2, \dots, R_m\}$ the set consisting of all resource types in the system.

A directed edge from process P_i to resource type R_j is denoted by $P_i \rightarrow R_j$ it signifies that process P_i has requested an instance of resource type R_j and is currently waiting for that resource.

A directed edge from resource type R_j to process P_i is denoted by $R_j \rightarrow P_i$ it signifies that an instance of resource type R_j has been allocated to process P_i .

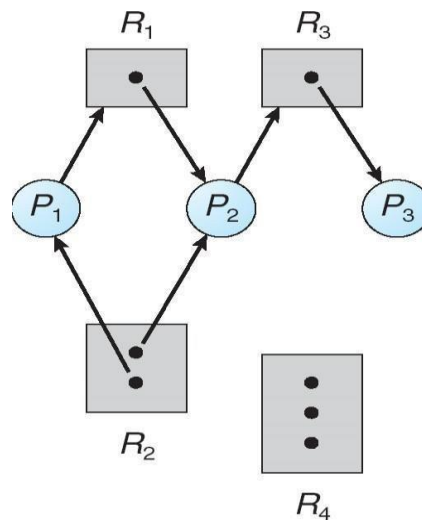
- A directed edge $P_i \rightarrow R_j$ is called a Request Edge.
- A directed edge $R_j \rightarrow P_i$ is called an Assignment Edge.

Pictorially each process P_i as a circle and each resource type R_j as a rectangle. Since resource type R_j may have more than one instance, each instance is represented as a dot within the rectangle.

A request edge points to only the rectangle R_j , whereas an assignment edge must also designate one of the dots in the rectangle.

When process P_i requests an instance of resource type R_j , a request edge is inserted in the resource-allocation graph. When this request can be fulfilled, the request edge is *instantaneously* transformed to an assignment edge. When the process no longer needs access to the resource, it releases the resource; as a result, the assignment edge is deleted.

The resource-allocation graph shown in Figure depicts the following situation.



The sets P , K and E :

- $P = \{P_1, P_2, P_3\}$
- $R = \{R_1, R_2, R_3, R_4\}$
- $E = \{P_1 \rightarrow R_1, P_2 \rightarrow R_3, R_1 \rightarrow P_2, R_2 \rightarrow P_2, R_2 \rightarrow P_1, R_3 \rightarrow P_3\}$

Resource instances:

- One instance of resource type R_1
- Two instances of resource type R_2
- One instance of resource type R_3
- Three instances of resource type R_4

Process states:

- Process P_1 is holding an instance of resource type R_2 and is waiting for an instance of resource type R_1 .
- Process P_2 is holding an instance of R_1 and an instance of R_2 and is waiting for an instance of R_3 .
- Process P_3 is holding an instance of R_3 .

If the graph does contain a cycle, then a deadlock may exist.

- If each resource type has exactly **one instance**, then a cycle implies that a deadlock has occurred. If the cycle involves only a set of resource types, each of which has only a single instance, then a deadlock has occurred. Each process involved in the cycle is deadlocked.
- If each resource type has **several instances**, then a cycle does not necessarily imply that a deadlock has occurred. In this case, a cycle in the graph is a necessary but not a sufficient condition for the existence of deadlock.

To illustrate this concept, the resource-allocation graph depicted in below figure:

Suppose that process P3 requests an instance of resource type R2. Since no resource instance is currently available, a request edge $P_3 \rightarrow R_2$ is added to the graph. At this point, two minimal cycles exist in the system:

1. $P_1 \rightarrow R_1 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$
2. $P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_2$

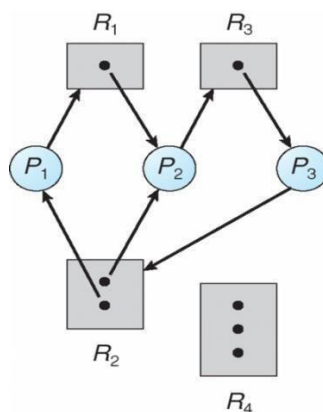


Figure: Resource-allocation graph with a deadlock.

Processes P1, P2, and P3 are deadlocked. Process P2 is waiting for the resource R3, which is held by process P3. Process P3 is waiting for either process P1 or process P2 to release resource R2. In addition, process P1 is waiting for process P2 to release resource R1.

Consider the resource-allocation graph in below Figure. In this example also have a cycle:

$P_1 \rightarrow R_1 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$

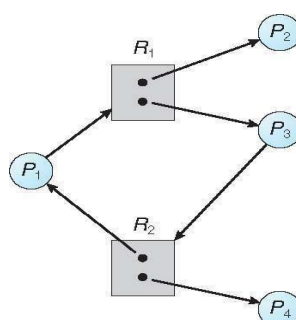


Figure: Resource-allocation graph with a cycle but no deadlock

However, there is no deadlock. Observe that process P4 may release its instance of resource type R2. That resource can then be allocated to P3, breaking the cycle.

METHODS FOR HANDLING DEADLOCKS

The deadlock problem can be handled in one of three ways:

1. Use a protocol to prevent or avoid deadlocks, ensuring that the system will never enter a deadlocked state.
2. Allow the system to enter a deadlocked state, detect it, and recover.
3. Ignore the problem altogether and pretend that deadlocks never occur in the system.

To ensure that deadlocks never occur, the system can use either deadlock prevention or a deadlock-avoidance scheme.

Deadlock prevention provides a set of methods for ensuring that at least one of the necessary conditions cannot hold. These methods prevent deadlocks by constraining how requests for resources can be made.

Deadlock-avoidance requires that the operating system be given in advance additional information concerning which resources a process will request and use during its lifetime. With this additional knowledge, it can decide for each request whether or not the process should wait. To decide whether the current request can be satisfied or must be delayed, the system must consider the resources currently available, the resources currently allocated to each process, and the future requests and releases of each process

If a system does not employ either a deadlock-prevention or a deadlock avoidance algorithm, then a deadlock situation may arise. In this environment, the system can provide an **algorithm** that examines the state of the system to determine whether a deadlock has occurred and an algorithm to recover from the deadlock.

In the absence of algorithms to detect and recover from deadlocks, then the system is in a deadlock state yet has no way of recognizing what has happened. In this case, the undetected deadlock will result in deterioration of the system's performance, because resources are being held by processes that cannot run and because more and more processes, as they make requests for resources, will enter a deadlocked state. Eventually, the system will stop functioning and will need to be restarted manually.

DEADLOCK PREVENTION

Deadlock can be prevented by ensuring that at least one of the four necessary conditions cannot hold.

Mutual Exclusion

- The mutual-exclusion condition must hold for non-sharable resources. Sharable resources, do not require mutually exclusive access and thus cannot be involved in a deadlock.
- Ex: Read-only files are example of a sharable resource. If several processes attempt to open a read-only file at the same time, they can be granted simultaneous access to the file. A process never needs to wait for a sharable resource.
- Deadlocks cannot prevent by denying the mutual-exclusion condition, because some resources are intrinsically non-sharable.

Hold and Wait

To ensure that the hold-and-wait condition never occurs in the system, then guarantee that, whenever a process requests a resource, it does not hold any other resources.

- One protocol that can be used requires each process to request and be allocated all its resources before it begins execution.
- Another protocol allows a process to request resources only when it has none. A process may request some resources and use them. Before it can request any additional resources, it must release all the resources that it is currently allocated.

Ex:

- Consider a process that copies data from a DVD drive to a file on disk, sorts the file, and then prints the results to a printer. If all resources must be requested at the beginning of the process, then the process must initially request the DVD drive, disk file, and printer. It will hold the printer for its entire execution, even though it needs the printer only at the end.
- The second method allows the process to request initially only the DVD drive and disk file. It copies from the DVD drive to the disk and then releases both the DVD drive and the disk file. The process must then again request the disk file and the printer. After copying the disk file to the printer, it releases these two resources and terminates.

The two main disadvantages of these protocols:

1. Resource utilization may be low, since resources may be allocated but unused for a long period.
2. Starvation is possible.

No Preemption

The third necessary condition for deadlocks is that there be no preemption of resources that have already been allocated.

To ensure that this condition does not hold, the following protocols can be used:

- If a process is holding some resources and requests another resource that cannot be immediately allocated to it, then all resources the process is currently holding are preempted.
- The preempted resources are added to the list of resources for which the process is waiting. The process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting.

If a process requests some resources, first check whether they are available. If they are, allocate them.

If they are not available, check whether they are allocated to some other process that is waiting for additional resources. If so, preempt the desired resources from the waiting process and allocate them to the requesting process.

If the resources are neither available nor held by a waiting process, the requesting process must wait. While it is waiting, some of its resources may be preempted, but only if another process requests them.

A process can be restarted only when it is allocated the new resources it is requesting and recovers any resources that were preempted while it was waiting.

Circular Wait

One way to ensure that this condition never holds is to impose a total ordering of all resource types and to require that each process requests resources in an increasing order of enumeration.

To illustrate, let $R = \{R_1, R_2, \dots, R_m\}$ be the set of resource types. Assign a unique integer number to each resource type, which allows to compare two resources and to determine whether one precedes another in ordering. Formally, it is defined as a one-to-one function

$F: R \rightarrow \mathbb{N}$, where $\mathbb{N}$ is the set of natural numbers.

Example: if the set of resource types R includes tape drives, disk drives, and printers, then the function F might be defined as follows:

$$F(\text{tape drive}) = 1$$

$$F(\text{disk drive}) = 5$$

$$F(\text{printer}) = 12$$

Now consider the following protocol to prevent deadlocks. Each process can request resources only in an increasing order of enumeration. That is, a process can initially request any number of instances of a resource type R_i . After that, the process can request instances of resource type R_j if and only if $F(R_j) > F(R_i)$.

DEADLOCK AVOIDANCE

- To avoid deadlocks an additional information is required about how resources are to be requested. With the knowledge of the complete sequence of requests and releases for each process, the system can decide for each request whether or not the process should wait in order to avoid a possible future deadlock
- Each request requires that in making this decision the system consider the resources currently available, the resources currently allocated to each process, and the future requests and releases of each process.
- The various algorithms that use this approach differ in the amount and type of information required. The simplest model requires that each process declare the *maximum number* of resources of each type that it may need. Given this a priori information, it is possible to construct an algorithm that ensures that the system will never enter a deadlocked state. Such an algorithm defines the ***deadlock-avoidance approach***.

Safe State

- **Safe state:** A state is safe if the system can allocate resources to each process (up to its maximum) in some order and still avoid a deadlock. A system is in a safe state only if there exists a safe sequence.
- **Safe sequence:** A sequence of processes $\langle P_1, P_2, \dots, P_n \rangle$ is a safe sequence for the current allocation state if, for each P_i , the resource requests that P_i can still make can be satisfied by the currently available resources plus the resources held by all P_j , with $j < i$.

In this situation, if the resources that P_i needs are not immediately available, then P_i can wait until all P_j have finished. When they have finished, P_i can obtain all of its needed resources, complete its designated task, return its allocated resources, and terminate. When P_i terminates, P_{i+1} can obtain its needed resources, and so on. If no such sequence exists, then the system state is said to be unsafe.

A safe state is not a deadlocked state. Conversely, a deadlocked state is an unsafe state. Not all unsafe states are deadlocks as shown in figure. An unsafe state may lead to a deadlock. As long as the state is safe, the operating system can avoid unsafe states

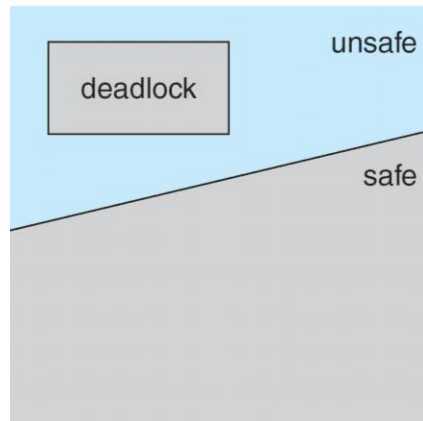


Figure: Safe, unsafe, and deadlocked state spaces.

Resource-Allocation-Graph Algorithm

- If a resource-allocation system has only one instance of each resource type, then a variant of the resource-allocation graph is used for deadlock avoidance.
- In addition to the request and assignment edges, a new type of edge is introduced, called a claim edge.
- A claim edge $P_i \rightarrow R_j$ indicates that process P_i may request resource R_j at some time in the future. This edge resembles a request edge in direction but is represented in the graph by a **dashed line**.
- When process P_i requests resource R_j , the claim edge $P_i \rightarrow R_j$ is converted to a request edge. When a resource R_j is released by P_i the assignment edge $R_j \rightarrow P_i$ is reconverted to a claim edge $P_i \rightarrow R_j$.

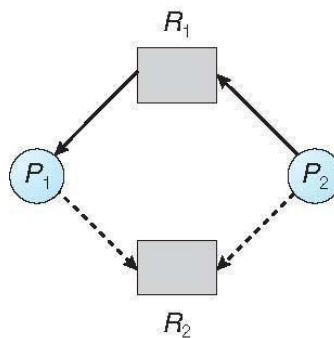


Figure: Resource-allocation graph for deadlock avoidance.

Note that the resources must be claimed a priori in the system. That is, before process P_i starts executing, all its claim edges must already appear in the resource-allocation graph.

We can relax this condition by allowing a claim edge $P_i \rightarrow R_j$ to be added to the graph only if all the edges associated with process P_i are claim edges.

Now suppose that process P_i requests resource R_j . The request can be granted only if converting the request edge $P_i \rightarrow R_j$ to an assignment edge $R_j \rightarrow P_i$ does not result in the formation of a cycle in the resource-allocation graph.

There is need to check for safety by using a cycle-detection algorithm. An algorithm for detecting a cycle in this graph requires an order of n^2 operations, where n is the number of processes in the system.

- If no cycle exists, then the allocation of the resource will leave the system in a safe state.
- If a cycle is found, then the allocation will put the system in an unsafe state. In that case, process P_i will have to wait for its requests to be satisfied.

To illustrate this algorithm, consider the resource-allocation graph as shown above. Suppose that P_2 requests R_2 . Although R_2 is currently free, we cannot allocate it to P_2 , since this action will create a cycle in the graph.

A cycle, indicates that the system is in an unsafe state. If P_1 requests R_2 , and P_2 requests R_1 , then a deadlock will occur.

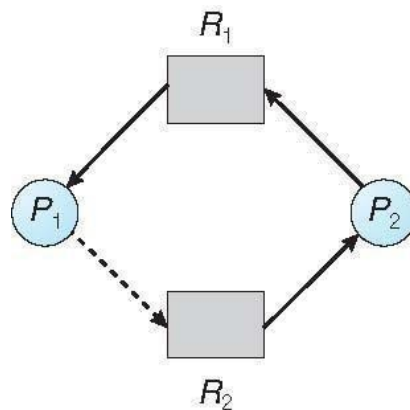


Figure: An unsafe state in a resource-allocation graph

Banker's Algorithm

The Banker's algorithm is applicable to a resource allocation system with multiple instances of each resource type.

- When a new process enters the system, it must declare the maximum number of instances of each resource type that it may need. This number may not exceed the total number of resources in the system.
- When a user requests a set of resources, the system must determine whether the allocation of these resources will leave the system in a safe state. If it will, the resources are allocated; otherwise, the process must wait until some other process releases enough resources.

To implement the banker's algorithm the following data structures are used.

Let n = number of processes, and m = number of resources types

Available: A vector of length m indicates the number of available resources of each type. If $\text{available}[j] = k$, there are k instances of resource type R_j available.

Max: An $n \times m$ matrix defines the maximum demand of each process. If $\text{Max}[i,j] = k$, then process P_i may request at most k instances of resource type R_j

Allocation: An $n \times m$ matrix defines the number of resources of each type currently allocated to each process. If $\text{Allocation}[i,j] = k$ then P_i is currently allocated k instances of R_j

Need: An $n \times m$ matrix indicates the remaining resource need of each process. If $\text{Need}[i,j] = k$, then P_i may need k more instances of R_j to complete its task.

$$\text{Need}[i,j] = \text{Max}[i,j] - \text{Allocation}[i,j]$$

Safety Algorithm

The algorithm for finding out whether or not a system is in a safe state. This algorithm can be described as follows:

1. Let **Work** and **Finish** be vectors of length m and n , respectively. Initialize:

Work = **Available**

Finish $[i] = \text{false}$ for $i = 0, 1, \dots, n-1$

2. Find an index i such that both:

(a) **Finish** $[i] = \text{false}$

(b) $\text{Need}_i \leq \text{Work}$

If no such i exists, go to step 4

3. **Work** = **Work** + **Allocation** $_i$

Finish $[i] = \text{true}$

go to step 2

4. If **Finish** $[i] == \text{true}$ for all i , then the system is in a safe state

This algorithm may require an order of $m \times n^2$ operations to determine whether a state is safe.



Resource-Request Algorithm

The algorithm for determining whether requests can be safely granted.

Let $Request_i$ be the request vector for process P_i . If $Request_i[j] = k$, then process P_i wants k instances of resource type R_j . When a request for resources is made by process P_i , the following actions are taken:

1. If $Request_i \leq Need_i$ go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim
2. If $Request_i \leq Available$, go to step 3. Otherwise P_i must wait, since resources are not available
3. Have the system pretend to allocate requested resources to P_i by modifying the state as follows:

$$\begin{aligned} Available &= Available - Request_i; \\ Allocation_i &= Allocation_i + Request_i; \\ Need_i &= Need_i - Request_i; \end{aligned}$$

If safe $\Rightarrow$ the resources are allocated to P_i

If unsafe $\Rightarrow P_i$ must wait, and the old resource-allocation state is restored

Example

Consider a system with five processes P_0 through P_4 and three resource types A , B , and C . Resource type A has ten instances, resource type B has five instances, and resource type C has seven instances. Suppose that, at time T_0 the following snapshot of the system has been taken:

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

The content of the matrix *Need* is defined to be *Max - Allocation*

	<u>Need</u>		
	A	B	C
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

The system is currently in a safe state. Indeed, the sequence $\langle P_1, P_3, P_4, P_2, P_0 \rangle$ satisfies the safety criteria.

Suppose now that process P_1 requests one additional instance of resource type A and two instances of resource type C, so $\text{Request}_1 = (1, 0, 2)$. Decide whether this request can be immediately granted.

Check that $\text{Request} \leq \text{Available}$

$$(1, 0, 2) \leq (3, 3, 2) \Rightarrow \text{true}$$

Then pretend that this request has been fulfilled, and the following new state is arrived.

	<u>Allocation</u>			<u>Need</u>			<u>Available</u>		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	4	3	2	3	0
P_1	3	0	2	0	2	0			
P_2	3	0	2	6	0	0			
P_3	2	1	1	0	1	1			
P_4	0	0	2	4	3	1			

Executing safety algorithm shows that sequence $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ satisfies safety requirement.

DEADLOCK DETECTION

If a system does not employ either a deadlock-prevention or a deadlock avoidance algorithm, then a deadlock situation may occur. In this environment, the system may provide:

- An algorithm that examines the state of the system to determine whether a deadlock has occurred
- An algorithm to recover from the deadlock

Single Instance of Each Resource Type

- If all resources have only a single instance, then define a deadlock detection algorithm that uses a variant of the resource-allocation graph, called a **wait-for** graph.
- This graph is obtained from the resource-allocation graph by removing the resource nodes and collapsing the appropriate edges.
- An edge from P_i to P_j in a wait-for graph implies that process P_i is waiting for process P_j to release a resource that P_i needs. An edge $P_i \rightarrow P_j$ exists in a wait-for graph if and only if the corresponding resource allocation graph contains two edges $P_i \rightarrow R_q$ and $R_q \rightarrow P_j$ for some resource R_q .

Example: In below Figure, a resource-allocation graph and the corresponding wait-for graph is presented.

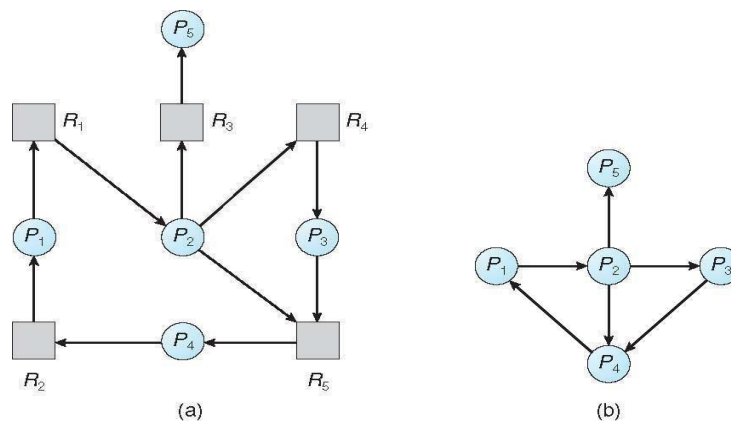


Figure: (a) Resource-allocation graph. (b) Corresponding wait-for graph.

- A deadlock exists in the system if and only if the wait-for graph contains a cycle. To detect deadlocks, the system needs to *maintain* the wait-for graph and periodically *invoke an algorithm* that searches for a cycle in the graph.
- An algorithm to detect a cycle in a graph requires an order of n^2 operations, where n is the number of vertices in the graph.

Several Instances of a Resource Type

A deadlock detection algorithm that is applicable to several instances of a resource type. The algorithm employs several time-varying data structures that are similar to those used in the banker's algorithm.

- **Available:** A vector of length m indicates the number of available resources of each type.
- **Allocation:** An $n \times m$ matrix defines the number of resources of each type currently allocated to each process.
- **Request:** An $n \times m$ matrix indicates the current request of each process. If $Request[i][j]$ equals k , then process P_i is requesting k more instances of resource type R_j .

Algorithm:

1. Let $Work$ and $Finish$ be vectors of length m and n , respectively Initialize:
 - (a) $Work = Available$
 - (b) For $i = 1, 2, \dots, n$, if $Allocation_i \neq 0$, then $Finish[i] = false$; otherwise, $Finish[i] = true$

2. Find an index i such that both:
 - (a) $Finish[i] == false$
 - (b) $Request_i \leq Work$

If no such i exists, go to step 4

3. $Work = Work + Allocation_i$
 $Finish[i] = true$
 go to step 2

4. If $Finish[i] == false$, for some i , $1 \leq i \leq n$, then the system is in deadlock state. Moreover, if $Finish[i] == false$, then P_i is deadlocked

Algorithm requires an order of $O(m \times n^2)$ operations to detect whether the system is in deadlocked state

Example of Detection Algorithm

Consider a system with five processes P_0 through P_4 and three resource types A , B , and C . Resource type A has seven instances, resource type B has two instances, and resource type C has six instances. Suppose that, at time T_0 , the following resource-allocation state:



	<u>Allocation</u>	<u>Request</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

After executing the algorithm, Sequence $\langle P_0, P_2, P_3, P_1, P_4 \rangle$ will result in $Finish[i] = \text{true}$ for all i

Suppose now that process P_2 makes one additional request for an instance of type C. The Request matrix is modified as follows:

	<u>Request</u>
	A B C
P_0	0 0 0
P_1	2 0 2
P_2	0 0 1
P_3	1 0 0
P_4	0 0 2

The system is now deadlocked. Although we can reclaim the resources held by process P_0 , the number of available resources is not sufficient to fulfill the requests of the other processes. Thus, a deadlock exists, consisting of processes P_1, P_2, P_3 , and P_4 .

Detection-Algorithm Usage

The detection algorithm can be invoked on two factors:

1. How *often* is a deadlock likely to occur?
2. How *many* processes will be affected by deadlock when it happens?

If deadlocks occur frequently, then the detection algorithm should be invoked frequently. Resources allocated to deadlocked processes will be idle until the deadlock can be broken.

If detection algorithm is invoked arbitrarily, there may be many cycles in the resource graph and so we would not be able to tell which of the many deadlocked processes “caused” the deadlock.

RECOVERY FROM DEADLOCK

The system recovers from the deadlock automatically. There are two options for breaking a deadlock one is simply to abort one or more processes to break the circular wait. The other is to preempt some resources from one or more of the deadlocked processes.

Process Termination

To eliminate deadlocks by aborting a process, use one of two methods. In both methods, the system reclaims all resources allocated to the terminated processes.

1. **Abort all deadlocked processes:** This method clearly will break the deadlock cycle, but at great expense; the deadlocked processes may have computed for a long time, and the results of these partial computations must be discarded and probably will have to be recomputed later.
2. **Abort one process at a time until the deadlock cycle is eliminated:** This method incurs considerable overhead, since after each process is aborted, a deadlock-detection algorithm must be invoked to determine whether any processes are still deadlocked.

If the partial termination method is used, then we must determine which deadlocked process (or processes) should be terminated. Many factors may affect which process is chosen, including:

1. What the priority of the process is
2. How long the process has computed and how much longer the process will compute before completing its designated task
3. How many and what types of resources the process has used.
4. How many more resources the process needs in order to complete
5. How many processes will need to be terminated?
6. Whether the process is interactive or batch

Resource Preemption

To eliminate deadlocks using resource preemption, we successively preempt some resources from processes and give these resources to other processes until the deadlock cycle is broken.

If preemption is required to deal with deadlocks, then three issues need to be addressed:

1. **Selecting a victim.** Which resources and which processes are to be preempted? As in process termination, we must determine the order of preemption to minimize cost. Cost factors may include such parameters as the number of resources a deadlocked process is holding and the amount of time the process has thus far consumed during its execution.
2. **Rollback.** If we preempt a resource from a process, what should be done with that process? Clearly, it cannot continue with its normal execution; it is missing some needed resource. We must roll back the process to some safe state and restart it from that state. Since it is difficult to determine what a safe state is, the simplest solution is a total rollback: abort the process and then restart it.



3. **Starvation.** How do we ensure that starvation will not occur? That is, how can we guarantee that resources will not always be preempted from the same process?

